

Microsoft Build 2026

GitHub Copilot アップデート Deep Dive

エージェントネイティブ開発のコントロールセンターへ

2026年6月2日 発表 / 一次情報ベースのまとめ（30分版・全48枚）

このセッションのゴール

- Microsoft Build 2026 で発表された **GitHub Copilot 関連アップデート** を体系的に理解する
- 単発の新機能ではなく、「ひとつのシステム」として何が変わったか を掴む
- 明日から **どう試せるか／どこを見ればよいか**（一次情報）まで持ち帰る

本資料は GitHub 公式ブログ・Microsoft 公式 `Build26-news` の **原文引用** に基づいて構成しています。
プレビュー機能は提供状況・名称が変更される場合があります。

アジェンダ（9テーマ）

#	テーマ	#	テーマ
0	背景：なぜ今エージェントか	5	Copilot SDK と CLI
1	GitHub Copilot アプリ	6	自動化とクラウドエージェント
2	Canvas と Agent Experience	7	パートナーエージェントアプリ
3	サンドボックス	8	AIモデル（MAI ファミリー）
4	コードレビュー	9	Microsoft IQ コンテキスト層

最後に **全体まとめ・導入の進め方・一次情報リンク** を扱います。

0. 背景

なぜ今「エージェント」なのか

0-1. 開発の現実：規模が爆発している

GitHub 上で、エージェント活用により開発のあらゆる活動が加速：

- リポジトリ作成・PR活動・API利用が **いずれも加速、鈍化の兆しなし**
- コミットは **前年比でほぼ倍増、月間 14億超**
- GitHub Actions は **週20億分超**

"commits nearly doubled year over year, crossing 1.4 billion per month, plus over 2 billion GitHub Actions minutes a week."

→ GitHub の最優先は **基盤のスケール、回復性・安定性の向上**

0-2. しかし「エージェント体験」は未成熟だった

エージェントで開発は速くなった一方、新たな課題が生まれた：

- ワークフローの **分断**、コンテキストスイッチの増加
- エージェント生成コードの **レビュー負荷** の増大
- 多くの開発ツールは **複数エージェントの並列指揮** を前提に作られていない
 - コンテキストがウィンドウ間に散乱し、何が動いているか見失う
 - PRに「何を試し・何を検証したか」の痕跡が残らない

"most developer tools were not designed for directing multiple agents in parallel."

0-3. Build 2026 の答え：ひとつのシステム

GitHub は エージェント・フロンティア向けのシステム を提示：

- **Copilot アプリ**：並列エージェントのコントロールセンター
- **Canvas**：作業を可視化・操作する双方向サーフェス
- **サンドボックス**：安全に実行・反復できる境界
- **コードレビュー / 自動化 / SDK / パートナー** が連携
- **MAI モデル・Microsoft IQ** が下支え

エージェントが多くの作業を担い、開発者は品質・ポリシー・デリバリーの統制を保つ

1. GitHub Copilot アプリ

エージェントネイティブなデスクトップ体験

1-1. 何が登場したか

GitHub 上に構築された、独立したデスクトップ体験

- エディタのサイドバーではなく、スタンドアロンのアプリ
- 既存プラン（**Copilot Pro / Pro+ / Business / Enterprise**）で **技術プレビュー**
- アイデア・既存 Issue・PR から開始し、複数エージェントを並列にオーケストレーション

"now in preview, brings agentic development to a native desktop experience – and a much wider audience"

1-2. My Work ダッシュボード

接続したリポジトリをまたいで、**進行中の作業を単一ビュー** で把握：

- アクティブな セッション
- Issue / Pull Request
- バックグラウンド自動化

利用イメージ：朝の時点で3つの作業が動いている

- 🐛 本番バグを調査するエージェント
- ✨ バックログのIssueを実装するエージェント
- 🔄 PRのレビューフィードバックに対応するエージェント

→ それぞれを **検査・軌道修正・テスト・マージ** できる

1-3. git worktree による並列セッション

各セッションは **独立した git worktree**（ブランチの実体コピー）上で動作：

- 複数エージェントが **互いに干渉せず** 並行作業
- worktree の **作成・後始末・ブランチ管理はアプリが自動処理**
- 手動セットアップ不要、ブランチのやりくりも不要

“Every session runs in its own git worktree, a real, isolated copy of your branch.”

→ 「並列でエージェントを走らせる」ことが現実的な運用になる

1-4. Agent Merge : PRをゴールまで運ぶ

Pull Request を レビュー・チェック・マージ まで運ぶ :

- CI を監視
- 必須レビュアー を追跡
- 失敗チェックに対処
- すべての条件が満たされるのを待つ

どこまで自動化するかは開発者が選択 :

選択肢	内容
CIをグリーンに戻す	失敗を解消して再実行
フィードバック対応	レビュー指摘に応える
条件達成でマージ	条件を満たしたらマージ

| "You decide what automation is enabled and what ships."

2. Canvas と Agent Experience (AX)

意図が「検査できる作業」になる場所

2-1. なぜ Canvas か：チャットの限界

- チャットは **指示** と **曖昧さの整理** には強力
- だがエージェントが実作業を始めると、スレッドは **決定・ログ・修正の長いスクロール** になる
- **作業そのものを可視化する場所** が必要

"once an agent starts doing real work, a chat thread becomes a long scroll of decisions, logs, and corrections."

2-2. Canvas とは

人とエージェントの双方向ワークサーフェス（bidirectional work surfaces）

表示できるもの：

- プラン（計画） / Pull Request
- ブラウザセッション / ターミナル
- デプロイメント / ダッシュボード / ワークフローの状態

動き方：

- エージェントが **作業しながら Canvas を更新**
- 開発者は同じ画面で **編集・並べ替え・承認・軌道修正**

2-3. Agent Experience (AX) の始まり

人とエージェントが共に操作するインターフェース = **AX** の幕開け：

	Chat	Canvas
目的	指示・議論・推論	作業の可視化・操作
方向性	主に 人 → エージェント	双方向（人 ⇄ エージェント）
表現	テキストの流れ	プラン/PR/ターミナル等の作業面

“Canvases are where that intent becomes visible work you can inspect, steer, and verify.”

→ 「提案を読む」から「**作業と一緒に操作する**」へ

3. サンドボックス

クラウド／ローカルの隔離実行環境

3-1. なぜサンドボックスが必要か

- コードを **提案するだけ** のエージェントは、人の作業負担が大きい
- 有効に働くには **実行・結果検査・テスト・反復** が必要
- ただし **本番に触れずに**

→ エージェントが安全に行動できる **境界づけられた場所 (bounded place)** を提供

"agents need to run code, inspect results, test changes, and iterate, without touching production."

3-2. ローカル vs クラウド

観点	ローカルサンドボックス	クラウドサンドボックス
実行場所	自分のマシン上の隔離環境	GitHubホストの隔離Linux環境
寿命	ローカル環境	エフェメラル（使い捨て）
アクセス	FS・ネットワーク・システム機能を制限	完全隔離
ポリシー	集中管理・強制が可能	組織が独自に定義
強み	低レイテンシ・手元で完結	デバイス非依存・リソース制約なし

- クラウドからは **どのデバイスでも** Copilotセッションをリモート制御で引き継ぎ可能

3-3. OSレベルの封じ込め：MXC

Microsoft Execution Containers (MXC) (プレビュー)

- OS自体が封じ込めを強制 する、エンタープライズグレードのサンドボックス
- 開発者・IT管理者が、よりシンプルに安全な環境を作成

関連：

- **OpenClaw on Windows**：OS強制の境界内でマルチステップ・ワークフローを実行
- **NVIDIA OpenShell**：MXCを使い、ポリシー管理・推論ルーティング・PII難読化を追加

"containment enforced by the operating system itself"

4. コードレビュー

エージェント出力にスケールするレビュー

4-1. 課題と基本方針

- エージェントが生む **PRが増える** ほど、レビューへの圧力が累積
- **Copilot code review** が、ノイズをふるい分ける適応的・エージェント的システムを提供
- 開発者は **本当に重要なところに集中** できる

“an adaptable, agentic system to filter through the noise”

4-2. 自社基準でのカスタマイズ

すべてのレビューに **自社の基準・社内システム・エンジニアリングコンテキスト** を反映：

- カスタムエージェントスキル
- MCP サーバー接続
- 設定可能な Actions ワークフロー

→「汎用的な指摘」ではなく「**自分たちのルールに沿ったレビュー**」へ

4-3. Medium tier review と専用スキル

Medium tier review（中位ティアレビュー）

- PRを より高い推論能力のモデル にルーティング → 精度・再現率を向上
- 管理者がリポジトリ単位で "low" / "medium" を設定
- 低リスクは軽量・低コスト、影響度の高いリポジトリは高性能モデル、と使い分け

専用スキル

- `/security-review` : セキュリティ重視の評価に特化した専用パス
- `/rubberduck` (GA) : 複数のモデルファミリーで実装を批評し、新規の問題を発見

4-4. Azure DevOps ネイティブ対応

Azure DevOps 上でも Copilot code review をネイティブに利用可能 に：

- ワンクリックレビュー
- インラインコメント
- コミット可能な修正提案
- 管理者は 任意のリポジトリ でレビューを有効化

GitHub と同じレビュー体験を、Azure DevOps のワークフローでも

5. Copilot SDK と CLI

One runtime, many surfaces

5-1. GitHub Copilot SDK

Copilotアプリを支えるのと同じエージェントランタイム を公開：

- GA言語：Node.js / TypeScript、Python、Go、.NET、Rust、Java
- 個別スタックを継ぎ接ぎせず、**同じ基盤の上で構築**

ユースケース例：

- 社内の **コード解析ツール**
- カスタムな **リリースノート生成ツール**
- サポートワークフローに **埋め込むエージェント**

|"it exposes the same agentic runtime that powers the Copilot app"

5-2. Copilot CLI の刷新

ターミナルで作業したい開発者向けに大幅刷新：

機能	内容
再設計TUI	<code>/experimental</code> モードで PR・Issue・gist にタブ切替アクセス
Voice mode	オンデバイス 音声テキスト変換（音声は端末外に出ない）
<code>/every</code>	繰り返しプロンプト・バックグラウンドタスクのスケジュール

「ターミナルに留まったまま」エージェント開発を完結できる

6. 自動化とクラウドエージェント

コードの外側まで担う

6-1. クラウド自動化

エージェントを次のように動かせる：

- スケジュール実行
- GitHub イベントへの応答
- Issue の起票 / コメントの投稿

許可モデル（重要）

- デフォルトは 各書き込みアクションの前に許可を求める
- 信頼を確立したら **autopilot**（自動操縦） に切り替え

"By default, the cloud agent asks permission before each write action. Switch to autopilot once you have established trust."

6-2. クラウドエージェントの守備範囲拡大

エンジニアリングは「コードを書いて終わり」ではない：

- Issue を **起票** する
- ディスカッションを **開始** する
- レビューアーに **返信** する

→ これら **すべてのステップ** をクラウドエージェントが処理できるように

|"Copilot cloud agent can now handle every one of those steps."

6-3. Memory++ と /chronicle

デバイスをまたぎ、時間を越えた継続性（continuity）を獲得：

- Memory++ / `/chronicle`
- アプリ・CLI・VS Code・GitHub 上で開始したセッションの
コンテキストを横断的に参照 できる

|"Query context from sessions started in the app, CLI, VS Code, or on GitHub."

→ 「どこで始めた作業でも、続きから扱える」

7. パートナーエージェントアプリ

GitHubを離れずにツールを使う

7-1. パートナーエージェントアプリとは

Partner-built agent apps が GitHub Copilot と統合：

- タスクの自動化・コード生成・コンテキスト分析・アクション実行を支援
- **GitHub を離れず**に お気に入りのツールを利用
- Issue を、ワークフローに合った **新しいエージェント**に割り当て
- 自社エージェント持ち込み用の **ウェイトリスト** も提供

"Use your favorite tools without leaving GitHub. Assign issues to new agents that fit your workflow."

7-2. 発表時点のパートナー

パートナー	一般的な領域
LaunchDarkly	フィーチャーフラグ / リリース管理
Amplitude	プロダクトアナリティクス
Sonar	コード品質 / 静的解析
Endor Labs	サプライチェーンセキュリティ
Octopus Deploy	デプロイ自動化
PagerDuty	インシデント管理 / オンコール
Miro	コラボレーション
Bright / Packfiles	—

※領域は一般的な製品カテゴリに基づく補足

8. AIモデル

MAI モデルファミリー

8-1. MAI-Code-1 (Copilot/VS Codeに搭載)

- 推論効率に優れたコーディングモデル
- GitHub 向けにチューニング
- Copilot および VS Code で利用可能

|"MAI-Code-1: our inference efficient coding model tuned for GitHub, is now available in Copilot and VS Code"

→ Copilot のコード生成・補完を、**専用最適化モデル** で底上げ

8-2. MAI モデルファミリー全体像

モデル	種別	ハイライト
MAI-Code-1	コーディング	GitHub向けチューニング・Copilot/VS Codeで利用可
MAI-Thinking-1	推論	ゼロ蒸留・35Bアクティブ・256Kコンテキスト
MAI-Image-2.5	画像	text/image-to-image・Arena上位
MAI-Transcribe 1.5	音声→文字	43言語・最先端精度
MAI-Voice-2	音声	15以上の追加言語・新音声

8-3. MAI-Thinking-1 の実力

- **ゼロ蒸留**：エンタープライズグレードでクリーン・商用ライセンス済みデータで一から学習
- **35B アクティブパラメータ / 256K コンテキスト** — 高効率・低トークンコスト
- ブラインドテストで **Sonnet 4.6** より好まれる
- SWE Bench Pro のコーディングで **Opus 4.6** に匹敵
- 複雑なマルチステップ指示・長文脈推論・コード生成に強い（Foundryでプライベートプレビュー）

"independent raters prefer it to Sonnet 4.6, and it matches Opus 4.6 on coding abilities on SWE Bench Pro"

8-4. 提供先の広がり

- MAIモデルは **Fireworks AI / Baseten / Open Router** でも提供
- **Fireworks AI は Foundry 上で GA**
 - エンタープライズガバナンス + Azureデータレジデンシー
 - どのモデルでも単一プラットフォーム体験

Microsoft Agent Platform の方針

"build your agent in GitHub, deploy it to Microsoft Foundry and optimize it automatically with models best suited for the job"

→ GitHubで構築 → Foundryへデプロイ → 用途最適なモデルで自動最適化

9. Microsoft IQ

エージェントに知識を接続する層

9-1. Microsoft IQ とは

世界知識 × 業務知識 をエージェントに接続するコンテキスト層：

- 今月 GA：GitHub Copilot / Microsoft Foundry / Copilot Studio をまたいで
- エージェントを両方の知識に **グラウンディング** (grounding)

|"a new context layer that grounds agents in both world knowledge and enterprise knowledge"

→ 出力がより賢く・状況認識的になる

9-2. IQ ファミリーの構成

層	役割
Work IQ	M365横断の職場知識（人・メール・文書・会議）。API は 6/16 GA
Fabric IQ	構造化業務データ上の共有セマンティック基盤
Foundry IQ	企業知識×ライブWebの検索プランニング
Web IQ	AIファースト・モデル非依存・MCPネイティブ、次点比 約2.5倍速

9-3. GitHub Copilot との関係（概念図）



→ Copilot のエージェントが、リポジトリを超えて M365・業務データ・Webを参照

まとめ

30分の振り返り

まとめ：3つのメッセージ

1. コントロールセンター化

Copilotアプリ + worktreeで、複数エージェントを並列に統制。Agent Mergeでマージまで。

2. ひとつのシステム

Canvas / サンドボックス / レビュー / 自動化 / SDK / パートナー が連携。

エージェントが作業を担い、**人は品質・ポリシー・デリバリーを統制。**

3. 基盤の強化

MAI-Code-1 が Copilot/VS Codeに、**Microsoft IQ** が知識を接続。

明日からの進め方（提案）

ステップ	アクション
1. 試す	Copilotアプリ（技術プレビュー）を既存プランで有効化
2. 並列体験	Issueを割り当て、worktreeで複数セッションを走らせる
3. 統制	コードレビューのlow/medium設定、サンドボックスのポリシー定義
4. 拡張	SDK/CLIで自社ツール化、パートナーアプリ・MCPを接続
5. 接続	Microsoft IQ で M365・業務データをエージェントに接続

一次情報ソース

GitHub Blog: *GitHub Copilot app: The agent-native desktop experience*

microsoft/Build26-news (OMB Developer Blog)

Microsoft Blog: <https://aka.ms/AA10pe80> / GitHub Docs: <https://docs.github.com/copilot>

リポジトリ内の 01 ~ 09 に各テーマの詳細あり

Thank you! — Q&A